7. Collaboration

There are language features in Python to help you construct well-defined APIs with clear interface boundaries. The Python community has established best practices that maximize the maintainability of code over time. There are also standard tools that ship with Python that enable large teams to work together across disparate environments.

Collaborating with others on Python programs requires being deliberate about how you write your code. Even if you're working on your own, chances are you'll be using code written by someone else via the standard library or open source packages. It's important to understand the mechanisms that make it easy to collaborate with other Python programmers.

Item 49: Write Docstrings for Every Function, Class, and Module

Documentation in Python is extremely important because of the dynamic nature of the language. Python provides built-in support for attaching documentation to blocks of code. Unlike many other languages, the documentation from a program's source code is directly accessible as the program runs.

For example, you can add documentation by providing a *docstring* immediately after the `def` statement of a function.

Click here to view code image

```
def palindrome(word):
    """Return True if the given word is a palindrome."""
    return word == word[::-1]
```

You can retrieve the docstring from within the Python program itself by accessing the function's `__doc__` special attribute.

Click here to view code image

```
print(repr(palindrome.__doc__))

>>>
'Return True if the given word is a palindrome.'
```

Docstrings can be attached to functions, classes, and modules. This connection is part of the process of compiling and running a Python program. Support for docstrings and the `__doc__` attribute has three consequences:

- The accessibility of documentation makes interactive development easier. You can inspect functions, classes, and modules to see their documentation by using the `help` built-in function. This makes the Python interactive interpreter (the Python "shell") and tools like IPython Notebook (http://ipython.org) a joy to use while you're developing algorithms, testing APIs, and writing code snippets.
- A standard way of defining documentation makes it easy to build tools that convert the text into more appealing formats (like HTML). This has led to excellent

documentation-generation tools for the Python community, such as Sphinx (http://sphinx-doc.org). It's also enabled community-funded sites like Read the Docs (https://readthedocs.org) that provide free hosting of beautiful-looking documentation for open source Python projects.

- Python's first-class, accessible, and good-looking documentation encourages people to write more documentation. The members of the Python community have a strong belief in the importance of documentation. There's an assumption that "good code" also means well-documented code. This means that you can expect most open source Python libraries to have decent documentation.

To participate in this excellent culture of documentation, you need to follow a few guidelines when you write docstrings. The full details are discussed online in PEP 257 (http://www.python.org/dev/peps/pep-0257/). There are a few best-practices you should be sure to follow.

Documenting Modules

Each module should have a top-level docstring. This is a string literal that is the first statement in a source file. It should use three double quotes (""""). The goal of this docstring is to introduce the module and its contents.

The first line of the docstring should be a single sentence describing the module's purpose. The paragraphs that follow should contain the details that all users of the module should know about its operation. The module docstring is also a jumping-off point where you can highlight important classes and functions found in the module.

Here's an example of a module docstring:

Click here to view code image

```
# words.py
#!/usr/bin/env python3
"""Library for testing words for various linguistic patterns.

Testing how words relate to each other can be tricky sometimes!
This module provides easy ways to determine when words you've
found have special properties.

Available functions:
- palindrome: Determine if a word is a palindrome.
- check_anagram: Determine if two words are anagrams.
…
"""

# …
```

If the module is a command-line utility, the module docstring is also a great place to put usage information for running the tool from the command-line.

Documenting Classes

Each class should have a class-level docstring. This largely follows the same pattern as the module-level docstring. The first line is the single-sentence purpose of the class. Paragraphs that follow discuss important details of the class's operation.

Important public attributes and methods of the class should be highlighted in the class-level docstring. It should also provide guidance to subclasses on how to properly interact with protected attributes (see Item 27: "Prefer Public Attributes Over Private Ones") and the superclass's methods.

Here's an example of a class docstring:

Click here to view code image

```
class Player(object):
    """Represents a player of the game.

    Subclasses may override the 'tick' method to provide
    custom animations for the player's movement depending
    on their power level, etc.

    Public attributes:
    - power: Unused power-ups (float between 0 and 1).
    - coins: Coins found during the level (integer).
    """

    # ...
```

Documenting Functions

Each public function and method should have a docstring. This follows the same pattern as modules and classes. The first line is the single-sentence description of what the function does. The paragraphs that follow should describe any specific behaviors and the arguments for the function. Any return values should be mentioned. Any exceptions that callers must handle as part of the function's interface should be explained.

Here's an example of a function docstring:

Click here to view code image

```
def find_anagrams(word, dictionary):
    """Find all anagrams for a word.

    This function only runs as fast as the test for
    membership in the 'dictionary' container. It will
    be slow if the dictionary is a list and fast if
    it's a set.

    Args:
        word: String of the target word.
        dictionary: Container with all strings that
            are known to be actual words.

    Returns:
        List of anagrams that were found. Empty if
        none were found.
    """
```

```
    # ...
```

There are also some special cases in writing docstrings for functions that are important to know.

- If your function has no arguments and a simple return value, a single sentence description is probably good enough.
- If your function doesn't return anything, it's better to leave out any mention of the return value instead of saying "returns None."
- If you don't expect your function to raise an exception during normal operation, don't mention that fact.
- If your function accepts a variable number of arguments (see Item 18: "Reduce Visual Noise with Variable Positional Arguments") or keyword-arguments (see Item 19: "Provide Optional Behavior with Keyword Arguments"), use `*args` and `**kwargs` in the documented list of arguments to describe their purpose.
- If your function has arguments with default values, those defaults should be mentioned (see Item 20: "Use `None` and Docstrings to Specify Dynamic Default Arguments").
- If your function is a generator (see Item 16: "Consider Generators Instead of Returning Lists"), then your docstring should describe what the generator yields when it's iterated.
- If your function is a coroutine (see Item 40: "Consider Coroutines to Run Many Functions Concurrently"), then your docstring should contain what the coroutine yields, what it expects to receive from `yield` expressions, and when it will stop iteration.

Note

Once you've written docstrings for your modules, it's important to keep the documentation up to date. The `doctest` built-in module makes it easy to exercise usage examples embedded in docstrings to ensure that your source code and its documentation don't diverge over time.

Things to Remember

- Write documentation for every module, class, and function using docstrings. Keep them up to date as your code changes.
- For modules: Introduce the contents of the module and any important classes or functions all users should know about.
- For classes: Document behavior, important attributes, and subclass behavior in the docstring following the `class` statement.
- For functions and methods: Document every argument, returned value, raised exception, and other behaviors in the docstring following the `def` statement.

Item 50: Use Packages to Organize Modules and Provide Stable APIs

As the size of a program's codebase grows, it's natural for you to reorganize its structure. You split larger functions into smaller functions. You refactor data structures into helper classes (see Item 22: "Prefer Helper Classes Over Bookkeeping with Dictionaries and Tuples"). You separate functionality into various modules that depend on each other.

At some point, you'll find yourself with so many modules that you need another layer in your program to make it understandable. For this purpose, Python provides *packages*. Packages are modules that contain other modules.

In most cases, packages are defined by putting an empty file named `__init__.py` into a directory. Once `__init__.py` is present, any other Python files in that directory will be available for import using a path relative to the directory. For example, imagine that you have the following directory structure in your program.

```
main.py
mypackage/__init__.py
mypackage/models.py
mypackage/utils.py
```

To import the `utils` module, you use the absolute module name that includes the package directory's name.

```
# main.py
from mypackage import utils
```

This pattern continues when you have package directories present within other packages (like `mypackage.foo.bar`).

Note

Python 3.4 introduces *namespace packages*, a more flexible way to define packages. Namespace packages can be composed of modules from completely separate directories, zip archives, or even remote systems. For details on how to use the advanced features of namespace packages, see PEP 420 (http://www.python.org/dev/peps/pep-0420/).

The functionality provided by packages has two primary purposes in Python programs.

Namespaces

The first use of packages is to help divide your modules into separate namespaces. This allows you to have many modules with the same filename but different absolute paths that are unique. For example, here's a program that imports attributes from two modules with the same name, `utils.py`. This works because the modules can be addressed by their absolute paths.

Click here to view code image

```
# main.py
from analysis.utils import log_base2_bucket
```

```
from frontend.utils import stringify

bucket = stringify(log_base2_bucket(33))
```

This approach breaks down when the functions, classes, or submodules defined in packages have the same names. For example, say you want to use the `inspect` function from both the `analysis.utils` and `frontend.utils` modules. Importing the attributes directly won't work because the second `import` statement will overwrite the value of `inspect` in the current scope.

Click here to view code image

```
# main2.py
from analysis.utils import inspect
from frontend.utils import inspect  # Overwrites!
```

The solution is to use the `as` clause of the `import` statement to rename whatever you've imported for the current scope.

Click here to view code image

```
# main3.py
from analysis.utils import inspect as analysis_inspect
from frontend.utils import inspect as frontend_inspect

value = 33
if analysis_inspect(value) == frontend_inspect(value):
    print('Inspection equal!')
```

The `as` clause can be used to rename anything you retrieve with the `import` statement, including entire modules. This makes it easy to access namespaced code and make its identity clear when you use it.

Note

Another approach for avoiding imported name conflicts is to always access names by their highest unique module name.

For the example above, you'd first `import analysis.utils` and `import frontend.utils`. Then, you'd access the `inspect` functions with the full paths of `analysis.utils.inspect` and `frontend.utils.inspect`.

This approach allows you to avoid the `as` clause altogether. It also makes it abundantly clear to new readers of the code where each function is defined.

Stable APIs

The second use of packages in Python is to provide strict, stable APIs for external consumers.

When you're writing an API for wider consumption, like an open source package (see Item 48: "Know Where to Find Community-Built Modules"), you'll want to provide stable functionality that doesn't change between releases. To ensure that happens, it's important to hide your internal code organization from external users. This enables you to refactor and improve your package's internal modules without breaking existing users.

Python can limit the surface area exposed to API consumers by using the `__all__` special attribute of a module or package. The value of `__all__` is a list of every name to export from the module as part of its public API. When consuming code does `from foo import *`, only the attributes in `foo.__all__` will be imported from `foo`. If `__all__` isn't present in `foo`, then only public attributes, those without a leading underscore, are imported (see Item 27: "Prefer Public Attributes Over Private Ones").

For example, say you want to provide a package for calculating collisions between moving projectiles. Here, I define the `models` module of `mypackage` to contain the representation of projectiles:

Click here to view code image

```
# models.py
__all__ = ['Projectile']

class Projectile(object):
    def __init__(self, mass, velocity):
        self.mass = mass
        self.velocity = velocity
```

I also define a `utils` module in `mypackage` to perform operations on the `Projectile` instances, such as simulating collisions between them.

Click here to view code image

```
# utils.py
from . models import Projectile

__all__ = ['simulate_collision']

def _dot_product(a, b):
    # …

def simulate_collision(a, b):
    # …
```

Now, I'd like to provide all of the public parts of this API as a set of attributes that are available on the `mypackage` module. This will allow downstream consumers to always import directly from `mypackage` instead of importing from `mypackage.models` or `mypackage.utils`. This ensures that the API consumer's code will continue to work even if the internal organization of `mypackage` changes (e.g., `models.py` is deleted).

To do this with Python packages, you need to modify the `__init__.py` file in the `mypackage` directory. This file actually becomes the contents of the `mypackage` module when it's imported. Thus, you can specify an explicit API for `mypackage` by limiting what you import into `__init__.py`. Since all of my internal modules already specify `__all__`, I can expose the public interface of `mypackage` by simply importing everything from the internal modules and updating `__all__` accordingly.

```
# __init__.py
__all__ = []
from . models import *
__all__ += models.__all__
from . utils import *
__all__ += utils.__all__
```

Here's a consumer of the API that directly imports from `mypackage` instead of accessing the inner modules:

Click here to view code image

```
# api_consumer.py
from mypackage import *

a = Projectile(1.5, 3)
b = Projectile(4, 1.7)
after_a, after_b = simulate_collision(a, b)
```

Notably, internal-only functions like `mypackage.utils._dot_product` will not be available to the API consumer on `mypackage` because they weren't present in `__all__`. Being omitted from `__all__` means they weren't imported by the `from mypackage import *` statement. The internal-only names are effectively hidden.

This whole approach works great when it's important to provide an explicit, stable API. However, if you're building an API for use between your own modules, the functionality of `__all__` is probably unnecessary and should be avoided. The namespacing provided by packages is usually enough for a team of programmers to collaborate on large amounts of code they control while maintaining reasonable interface boundaries.

Beware of `import *`

Import statements like `from x import y` are clear because the source of `y` is explicitly the `x` package or module. Wildcard imports like `from foo import *` can also be useful, especially in interactive Python sessions. However, wildcards make code more difficult to understand.

- `from foo import *` hides the source of names from new readers of the code. If a module has multiple `import *` statements, you'll need to check all of the referenced modules to figure out where a name was defined.
- Names from `import *` statements will overwrite any conflicting names within the containing module. This can lead to strange bugs caused by accidental interactions between your code and overlapping names from multiple `import *` statements.

The safest approach is to avoid `import *` in your code and explicitly import names with the `from x import y` style.

Things to Remember

- Packages in Python are modules that contain other modules. Packages allow you to organize your code into separate, non-conflicting namespaces with unique absolute module names.
- Simple packages are defined by adding an `__init__.py` file to a directory that contains other source files. These files become the child modules of the directory's package. Package directories may also contain other packages.
- You can provide an explicit API for a module by listing its publicly visible names in

its `__all__` special attribute.

- You can hide a package's internal implementation by only importing public names in the package's `__init__.py` file or by naming internal-only members with a leading underscore.
- When collaborating within a single team or on a single codebase, using `__all__` for explicit APIs is probably unnecessary.

Item 51: Define a Root `Exception` to Insulate Callers from APIs

When you're defining a module's API, the exceptions you throw are just as much a part of your interface as the functions and classes you define (see Item 14: "Prefer Exceptions to Returning `None`").

Python has a built-in hierarchy of exceptions for the language and standard library. There's a draw to using the built-in exception types for reporting errors instead of defining your own new types. For example, you could raise a `ValueError` exception whenever an invalid parameter is passed to your function.

Click here to view code image

```
def determine_weight(volume, density):
    if density <= 0:
        raise ValueError('Density must be positive')
    # …
```

In some cases, using `ValueError` makes sense, but for APIs it's much more powerful to define your own hierarchy of exceptions. You can do this by providing a root `Exception` in your module. Then, have all other exceptions raised by that module inherit from the root exception.

Click here to view code image

```
# my_module.py
class Error(Exception):
    """Base-class for all exceptions raised by this module."""

class InvalidDensityError(Error):
    """There was a problem with a provided density value."""
```

Having a root exception in a module makes it easy for consumers of your API to catch all of the exceptions that you raise on purpose. For example, here a consumer of your API makes a function call with a `try`/`except` statement that catches your root exception:

Click here to view code image

```
try:
    weight = my_module.determine_weight(1, -1)
except my_module.Error as e:
    logging.error('Unexpected error: %s', e)
```

This `try`/`except` prevents your API's exceptions from propagating too far upward and breaking the calling program. It insulates the calling code from your API. This insulation has three helpful effects.

First, root exceptions let callers understand when there's a problem with their usage of your API. If callers are using your API properly, they should catch the various exceptions that you deliberately raise. If they don't handle such an exception, it will propagate all the way up to the insulating `except` block that catches your module's root exception. That block can bring the exception to the attention of the API consumer, giving them a chance to add proper handling of the exception type.

Click here to view code image

```
try:
    weight = my_module.determine_weight(1, -1)
except my_module.InvalidDensityError:
    weight = 0
except my_module.Error as e:
    logging.error('Bug in the calling code: %s', e)
```

The second advantage of using root exceptions is that they can help find bugs in your API module's code. If your code only deliberately raises exceptions that you define within your module's hierarchy, then all other types of exceptions raised by your module must be the ones that you didn't intend to raise. These are bugs in your API's code.

Using the `try`/`except` statement above will not insulate API consumers from bugs in your API module's code. To do that, the caller needs to add another `except` block that catches Python's base `Exception` class. This allows the API consumer to detect when there's a bug in the API module's implementation that needs to be fixed.

Click here to view code image

```
try:
    weight = my_module.determine_weight(1, -1)
except my_module.InvalidDensityError:
    weight = 0
except my_module.Error as e:
    logging.error('Bug in the calling code: %s', e)
except Exception as e:
    logging.error('Bug in the API code: %s', e)
    raise
```

The third impact of using root exceptions is future-proofing your API. Over time, you may want to expand your API to provide more specific exceptions in certain situations. For example, you could add an `Exception` subclass that indicates the error condition of supplying negative densities.

Click here to view code image

```
# my_module.py
class NegativeDensityError(InvalidDensityError):
    """A provided density value was negative."""

def determine_weight(volume, density):
    if density < 0:
        raise NegativeDensityError
```

The calling code will continue to work exactly as before because it already catches `InvalidDensityError` exceptions (the parent class of `NegativeDensityError`). In the future, the caller could decide to special-case the new type of exception and change its behavior accordingly.

Click here to view code image

```
try:
    weight = my_module.determine_weight(1, -1)
except my_module.NegativeDensityError as e:
    raise ValueError('Must supply non-negative density') from e
except my_module.InvalidDensityError:
    weight = 0
except my_module.Error as e:
    logging.error('Bug in the calling code: %s', e)
except Exception as e:
    logging.error('Bug in the API code: %s', e)
    raise
```

You can take API future-proofing further by providing a broader set of exceptions directly below the root exception. For example, imagine you had one set of errors related to calculating weights, another related to calculating volume, and a third related to calculating density.

Click here to view code image

```
# my_module.py
class WeightError(Error):
    """Base-class for weight calculation errors."""

class VolumeError(Error):
    """Base-class for volume calculation errors."""

class DensityError(Error):
    """Base-class for density calculation errors."""
```

Specific exceptions would inherit from these general exceptions. Each intermediate exception acts as its own kind of root exception. This makes it easier to insulate layers of calling code from API code based on broad functionality. This is much better than having all callers catch a long list of very specific `Exception` subclasses.

Things to Remember

- Defining root exceptions for your modules allows API consumers to insulate themselves from your API.
- Catching root exceptions can help you find bugs in code that consumes an API.
- Catching the Python `Exception` base class can help you find bugs in API implementations.
- Intermediate root exceptions let you add more specific types of exceptions in the future without breaking your API consumers.

Item 52: Know How to Break Circular Dependencies

Inevitably, while you're collaborating with others, you'll find a mutual interdependency between modules. It can even happen while you work by yourself on the various parts of a single program.

For example, say you want your GUI application to show a dialog box for choosing where to save a document. The data displayed by the dialog could be specified through

arguments to your event handlers. But the dialog also needs to read global state, like user preferences, to know how to render properly.

Here, I define a dialog that retrieves the default document save location from global preferences:

Click here to view code image

```
# dialog.py
import app

class Dialog(object):
    def __init__(self, save_dir):
        self.save_dir = save_dir
    # …

save_dialog = Dialog(app.prefs.get('save_dir'))

def show():
    # …
```

The problem is that the `app` module that contains the `prefs` object also imports the `dialog` class in order to show the dialog on program start.

```
# app.py
import dialog

class Prefs(object):
    # …
    def get(self, name):
        # …

prefs = Prefs()
dialog.show()
```

It's a circular dependency. If you try to use the `app` module from your main program, you'll get an exception when you import it.

Click here to view code image

```
Traceback (most recent call last):
  File "main.py", line 4, in <module>
    import app
  File "app.py", line 4, in <module>
    import dialog
  File "dialog.py", line 16, in <module>
    save_dialog = Dialog(app.prefs.get('save_dir'))
AttributeError: 'module' object has no attribute 'prefs'
```

To understand what's happening here, you need to know the details of Python's import machinery. When a module is imported, here's what Python actually does in depth-first order:

1. Searches for your module in locations from `sys.path`
2. Loads the code from the module and ensures that it compiles
3. Creates a corresponding empty module object
4. Inserts the module into `sys.modules`

5. Runs the code in the module object to define its contents

The problem with a circular dependency is that the attributes of a module aren't defined until the code for those attributes has executed (after step #5). But the module can be loaded with the `import` statement immediately after it's inserted into `sys.modules` (after step #4).

In the example above, the `app` module imports `dialog` before defining anything. Then, the `dialog` module imports `app`. Since `app` still hasn't finished running—it's currently importing `dialog`—the `app` module is just an empty shell (from step #4). The `AttributeError` is raised (during step #5 for `dialog`) because the code that defines `prefs` hasn't run yet (step #5 for `app` isn't complete).

The best solution to this problem is to refactor your code so that the `prefs` data structure is at the bottom of the dependency tree. Then, both `app` and `dialog` can import the same utility module and avoid any circular dependencies. But such a clear division isn't always possible or could require too much refactoring to be worth the effort.

There are three other ways to break circular dependencies.

Reordering Imports

The first approach is to change the order of imports. For example, if you import the `dialog` module toward the bottom of the `app` module, after its contents have run, the `AttributeError` goes away.

```
# app.py
class Prefs(object):
    # …

prefs = Prefs()

import dialog  # Moved
dialog.show()
```

This works because, when the `dialog` module is loaded late, its recursive import of `app` will find that `app.prefs` has already been defined (step #5 is mostly done for `app`).

Although this avoids the `AttributeError`, it goes against the PEP 8 style guide (see Item 2: "Follow the PEP 8 Style Guide"). The style guide suggests that you always put imports at the top of your Python files. This makes your module's dependencies clear to new readers of the code. It also ensures that any module you depend on is in scope and available to all the code in your module.

Having imports later in a file can be brittle and can cause small changes in the ordering of your code to break the module entirely. Thus, you should avoid import reordering to solve your circular dependency issues.

Import, Configure, Run

A second solution to the circular imports problem is to have your modules minimize side effects at import time. You have your modules only define functions, classes, and constants. You avoid actually running any functions at import time. Then, you have each module provide a `configure` function that you call once all other modules have finished importing. The purpose of `configure` is to prepare each module's state by accessing the attributes of other modules. You run `configure` after all modules have been imported (step #5 is complete), so all attributes must be defined.

Here, I redefine the `dialog` module to only access the `prefs` object when `configure` is called:

Click here to view code image

```
# dialog.py
import app

class Dialog(object):
    # …

save_dialog = Dialog()

def show():
    # …

def configure():
    save_dialog.save_dir = app.prefs.get('save_dir')
```

I also redefine the `app` module to not run any activities on import.

```
# app.py
import dialog

class Prefs(object):
    # …

prefs = Prefs()

def configure():
    # …
```

Finally, the `main` module has three distinct phases of execution: import everything, `configure` everything, and run the first activity.

```
# main.py
import app
import dialog

app.configure()
dialog.configure()

dialog.show()
```

This works well in many situations and enables patterns like *dependency injection*. But sometimes it can be difficult to structure your code so that an explicit `configure` step is possible. Having two distinct phases within a module can also make your code harder to

read because it separates the definition of objects from their configuration.

Dynamic Import

The third—and often simplest—solution to the circular imports problem is to use an `import` statement within a function or method. This is called a *dynamic import* because the module import happens while the program is running, not while the program is first starting up and initializing its modules.

Here, I redefine the `dialog` module to use a dynamic import. The `dialog.show` function imports the `app` module at runtime instead of the `dialog` module importing `app` at initialization time.

Click here to view code image

```
# dialog.py
class Dialog(object):
    # …

save_dialog = Dialog()

def show():
    import app  # Dynamic import
    save_dialog.save_dir = app.prefs.get('save_dir')
    # …
```

The `app` module can now be the same as it was in the original example. It imports `dialog` at the top and calls `dialog.show` at the bottom.

```
# app.py
import dialog

class Prefs(object):
    # …
prefs = Prefs()
dialog.show()
```

This approach has a similar effect to the import, configure, and run steps from before. The difference is that this requires no structural changes to the way the modules are defined and imported. You're simply delaying the circular import until the moment you must access the other module. At that point, you can be pretty sure that all other modules have already been initialized (step #5 is complete for everything).

In general, it's good to avoid dynamic imports like this. The cost of the `import` statement is not negligible and can be especially bad in tight loops. By delaying execution, dynamic imports also set you up for surprising failures at runtime, such as `SyntaxError` exceptions long after your program has started running (see Item 56: "Test Everything with `unittest`" for how to avoid that). However, these downsides are often better than the alternative of restructuring your entire program.

Things to Remember

- Circular dependencies happen when two modules must call into each other at import time. They can cause your program to crash at startup.

- The best way to break a circular dependency is refactoring mutual dependencies into a separate module at the bottom of the dependency tree.
- Dynamic imports are the simplest solution for breaking a circular dependency between modules while minimizing refactoring and complexity.

Item 53: Use Virtual Environments for Isolated and Reproducible Dependencies

Building larger and more complex programs often leads you to rely on various packages from the Python community (see Item 48: "Know Where to Find Community-Built Modules"). You'll find yourself running `pip` to install packages like `pytz`, `numpy`, and many others.

The problem is that, by default, `pip` installs new packages in a global location. That causes all Python programs on your system to be affected by these installed modules. In theory, this shouldn't be an issue. If you install a package and never `import` it, how could it affect your programs?

The trouble comes from transitive dependencies: the packages that the packages you install depend on. For example, you can see what the `Sphinx` package depends on after installing it by asking `pip`.

Click here to view code image

```
$ pip3 show Sphinx
–
Name: Sphinx
Version: 1.2.2
Location: /usr/local/lib/python3.4/site-packages
Requires: docutils, Jinja2, Pygments
```

If you install another package like `flask`, you can see that it, too, depends on the `Jinja2` package.

Click here to view code image

```
$ pip3 show flask
–
Name: Flask
Version: 0.10.1
Location: /usr/local/lib/python3.4/site-packages
Requires: Werkzeug, Jinja2, itsdangerous
```

The conflict arises as `Sphinx` and `flask` diverge over time. Perhaps right now they both require the same version of `Jinja2` and everything is fine. But six months or a year from now, `Jinja2` may release a new version that makes breaking changes to users of the library. If you update your global version of `Jinja2` with `pip install --upgrade`, you may find that `Sphinx` breaks while `flask` keeps working.

The cause of this breakage is that Python can only have a single global version of a module installed at a time. If one of your installed packages must use the new version and another package must use the old version, your system isn't going to work properly.

Such breakage can even happen when package maintainers try their best to preserve API

compatibility between releases (see Item 50: “Use Packages to Organize Modules and Provide Stable APIs”). New versions of a library can subtly change behaviors that API-consuming code relies on. Users on a system may upgrade one package to a new version but not others, which could dependencies. There’s a constant risk of the ground moving beneath your feet.

These difficulties are magnified when you collaborate with other developers who do their work on separate computers. It’s reasonable to assume that the versions of Python and global packages they have installed on their machines will be slightly different than your own. This can cause frustrating situations where a codebase works perfectly on one programmer’s machine and is completely broken on another’s.

The solution to all of these problems is a tool called `pyvenv`, which provides *virtual environments*. Since Python 3.4, the `pyvenv` command-line tool is available by default along with the Python installation (it’s also accessible with `python -m venv`). Prior versions of Python require installing a separate package (with `pip install virtualenv`) and using a command-line tool called `virtualenv`.

`pyvenv` allows you to create isolated versions of the Python environment. Using `pyvenv`, you can have many different versions of the same package installed on the same system at the same time without conflicts. This lets you work on many different projects and use many different tools on the same computer.

`pyvenv` does this by installing explicit versions of packages and their dependencies into completely separate directory structures. This makes it possible to reproduce a Python environment that you know will work with your code. It’s a reliable way to avoid surprising breakages.

The `pyvenv` Command

Here’s a quick tutorial on how to use `pyvenv` effectively. Before using the tool, it’s important to note the meaning of the `python3` command-line on your system. On my computer, `python3` is located in the `/usr/local/bin` directory and evaluates to version 3.4.2 (see Item 1: “Know Which Version of Python You’re Using”).

```
$ which python3
/usr/local/bin/python3
$ python3 –version
Python 3.4.2
```

To demonstrate the setup of my environment, I can test that running a command to import the `pytz` module doesn’t cause an error. This works because I already have the `pytz` package installed as a global module.

```
$ python3 -c 'import pytz'
$
```

Now, I use `pyvenv` to create a new virtual environment called `myproject`. Each virtual environment must live in its own unique directory. The result of the command is a tree of directories and files.

Click here to view code image

```
$ pyvenv /tmp/myproject
$ cd /tmp/myproject
$ ls
bin     include     lib     pyvenv.cfg
```

To start using the virtual environment, I use the `source` command from my shell on the `bin/activate` script. `activate` modifies all of my environment variables to match the virtual environment. It also updates my command-line prompt to include the virtual environment name (`'myproject'`) to make it extremely clear what I'm working on.

```
$ source bin/activate
(myproject)$
```

After activation, you can see that the path to the `python3` command-line tool has moved to within the virtual environment directory.

Click here to view code image

```
(myproject)$ which python3
/tmp/myproject/bin/python3
(myproject)$ ls -l /tmp/myproject/bin/python3
… -> /tmp/myproject/bin/python3.4
(myproject)$ ls -l /tmp/myproject/bin/python3.4
… -> /usr/local/bin/python3.4
```

This ensures that changes to the outside system will not affect the virtual environment. Even if the outer system upgrades its default `python3` to version 3.5, my virtual environment will still explicitly point to version 3.4.

The virtual environment I created with `pyvenv` starts with no packages installed except for `pip` and `setuptools`. Trying to use the `pytz` package that was installed as a global module in the outside system will fail because it's unknown to the virtual environment.

Click here to view code image

```
(myproject)$ python3 -c 'import pytz'
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ImportError: No module named 'pytz'
```

I can use `pip` to install the `pytz` module into my virtual environment.

Click here to view code image

```
(myproject)$ pip3 install pytz
```

Once it's installed, I can verify that it's working with the same test import command.

Click here to view code image

```
(myproject)$ python3 -c 'import pytz'
(myproject)$
```

When you're done with a virtual environment and want to go back to your default system, you use the `deactivate` command. This restores your environment to the system defaults, including the location of the `python3` command-line tool.

```
(myproject)$ deactivate
$ which python3
/usr/local/bin/python3
```

If you ever want to work in the `myproject` environment again, you can just run

`source bin/activate` in the directory like before.

Reproducing Dependencies

Once you have a virtual environment, you can continue installing packages with `pip` as you need them. Eventually, you may want to copy your environment somewhere else. For example, say you want to reproduce your development environment on a production server. Or maybe you want to clone someone else's environment on your own machine so you can run their code.

`pyvenv` makes these situations easy. You can use the `pip freeze` command to save all of your explicit package dependencies into a file. By convention, this file is named `requirements.txt`.

Click here to view code image

```
(myproject)$ pip3 freeze > requirements.txt
(myproject)$ cat requirements.txt
numpy==1.8.2
pytz==2014.4
requests==2.3.0
```

Now, imagine that you'd like to have another virtual environment that matches the `myproject` environment. You can create a new directory like before using `pyvenv` and `activate` it.

```
$ pyvenv /tmp/otherproject
$ cd /tmp/otherproject
$ source bin/activate
(otherproject)$
```

The new environment will have no extra packages installed.

```
(otherproject)$ pip3 list
pip (1.5.6)
setuptools (2.1)
```

You can install all of the packages from the first environment by running `pip install` on the `requirements.txt` that you generated with the `pip freeze` command.

Click here to view code image

```
(otherproject)$ pip3 install -r /tmp/myproject/requirements.txt
```

This command will crank along for a little while as it retrieves and installs all of the packages required to reproduce the first environment. Once it's done, listing the set of installed packages in the second virtual environment will produce the same list of dependencies found in the first virtual environment.

```
(otherproject)$ pip list
numpy (1.8.2)
pip (1.5.6)
pytz (2014.4)
requests (2.3.0)
setuptools (2.1)
```

Using a `requirements.txt` file is ideal for collaborating with others through a revision control system. You can commit changes to your code at the same time you

update your list of package dependencies, ensuring that they move in lockstep.

The gotcha with virtual environments is that moving them breaks everything because all of the paths, like `python3`, are hard-coded to the environment's install directory. But that doesn't matter. The whole purpose of virtual environments is to make it easy to reproduce the same setup. Instead of moving a virtual environment directory, just `freeze` the old one, create a new one somewhere else, and reinstall everything from the `requirements.txt` file.

Things to Remember

- Virtual environments allow you to use `pip` to install many different versions of the same package on the same machine without conflicts.
- Virtual environments are created with `pyvenv`, enabled with `source bin/activate`, and disabled with `deactivate`.
- You can dump all of the requirements of an environment with `pip freeze`. You can reproduce the environment by supplying the `requirements.txt` file to `pip install -r`.
- In versions of Python before 3.4, the `pyvenv` tool must be downloaded and installed separately. The command-line tool is called `virtualenv` instead of `pyvenv`.